Turing Machines

Turing machine

The mathematical models (FAs, TGs, PDAs) that have been discussed so far can decide whether a string is accepted or not by them *i.e*. these models are language identifiers. However, there are still some languages which can't be accepted by them *e.g.* there does not exist any FA or TG or PDA accepting any non-CFLs.

Alan Mathison Turing developed the machines called Turing machines, which accept some non-CFLs as well, in addition to CFLs.

- We now turn to the most powerful kind of automaton we will study: the Turing machine. Although it is only slightly more complicated than a finite-state machine, a Turing machine can do much more. It is, in fact, so powerful that it is the accepted champion of automata. No other, more powerful model exists.

- The TM is the strongest of computational mechanisms, the automaton of steel. But like all superheroes it does have one weakness, revealed in this chapter: it can get stuck in an infinite loop.

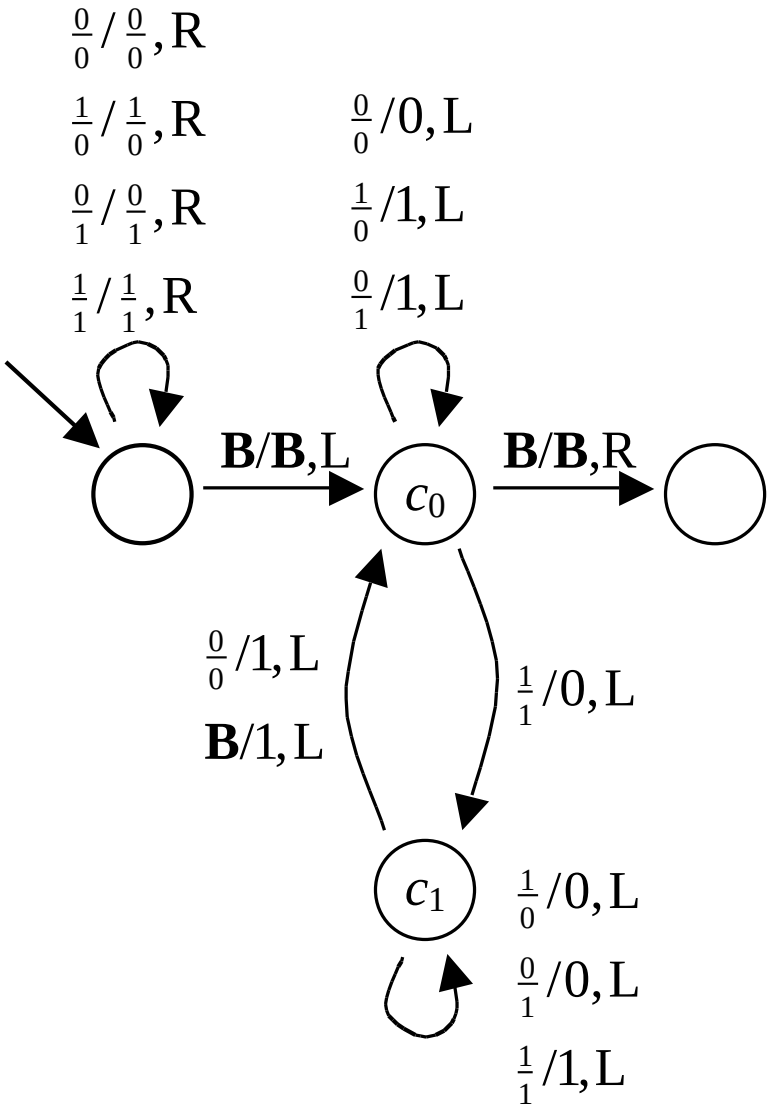
The *add* Function

- Binary addition, using stacked inputs of unbounded length
- For example, 27 + 22 = 49:

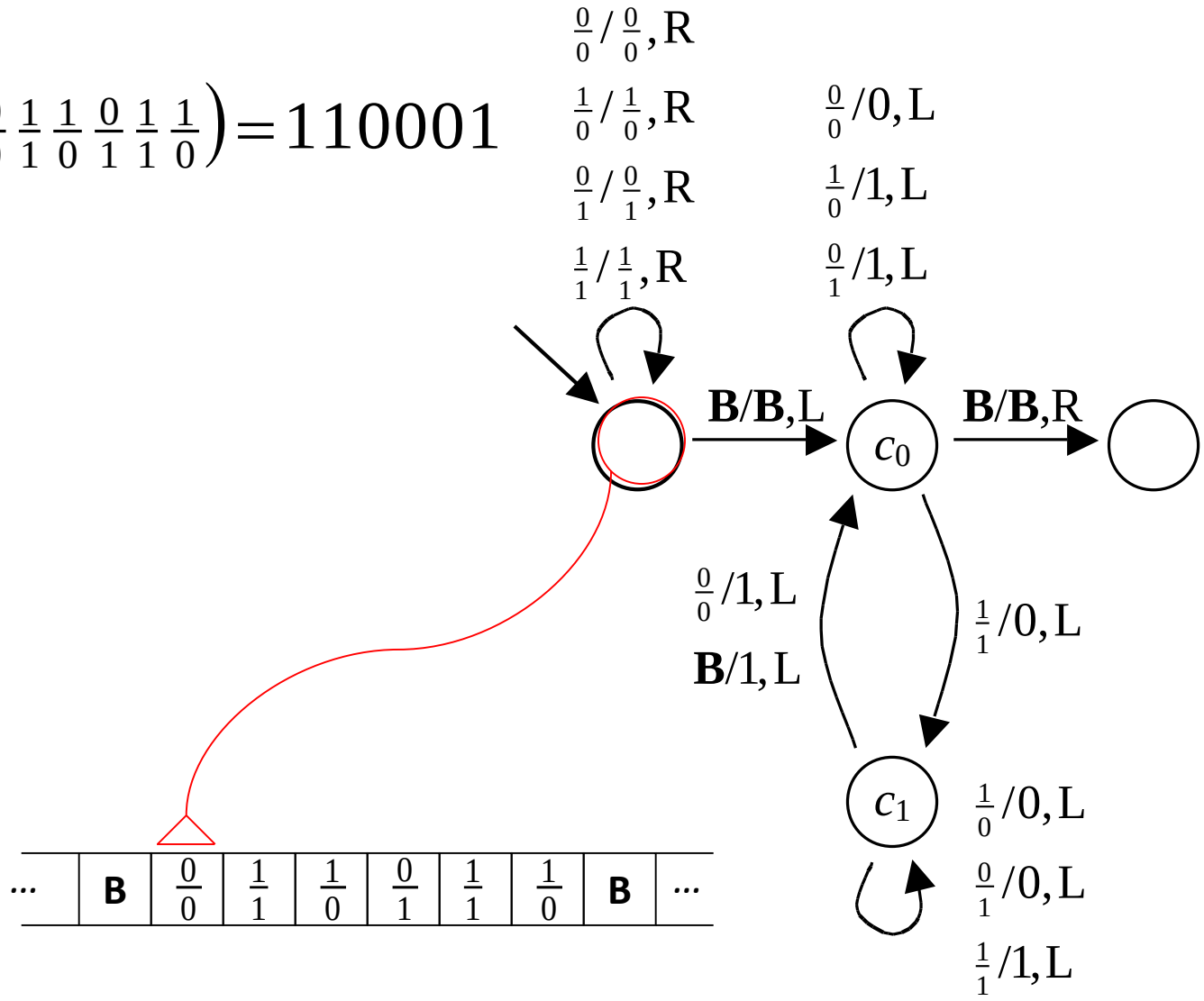
$$add\left(\begin{matrix}0&1&1&0&1&1\\0&1&0&1&1&0\end{matrix}\right)=110001$$

- The *add* function is Turing computable…

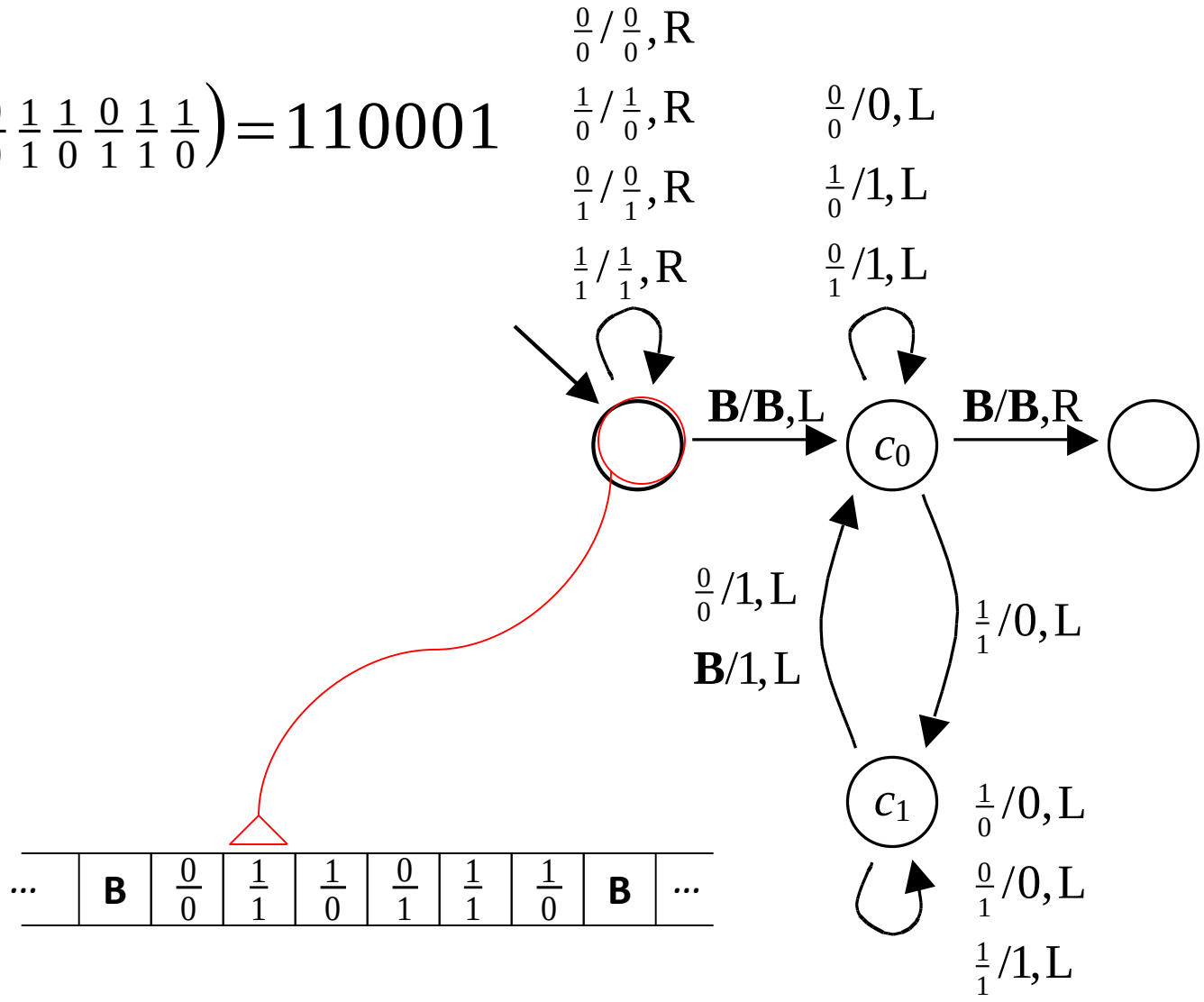
add


$\frac{0}{0}/\frac{0}{0},\mathrm{R}$
$\frac{1}{0}/\frac{1}{0},\mathrm{R}$
$\frac{0}{1}/\frac{0}{1},\mathrm{R}$
$\frac{1}{1}/\frac{1}{1},\mathrm{R}$
$\frac{0}{0}/0,\mathrm{L}$
$\frac{1}{0}/1,\mathrm{L}$
$\frac{0}{1}/1,\mathrm{L}$
B/B,L
B/B,R
c_0
$\frac{0}{0}/1,\mathrm{L}$
B/1,L
$\frac{1}{1}/0,\mathrm{L}$
c_1
$\frac{1}{0}/0,\mathrm{L}$
$\frac{0}{1}/0,\mathrm{L}$
$\frac{1}{1}/1,\mathrm{L}$

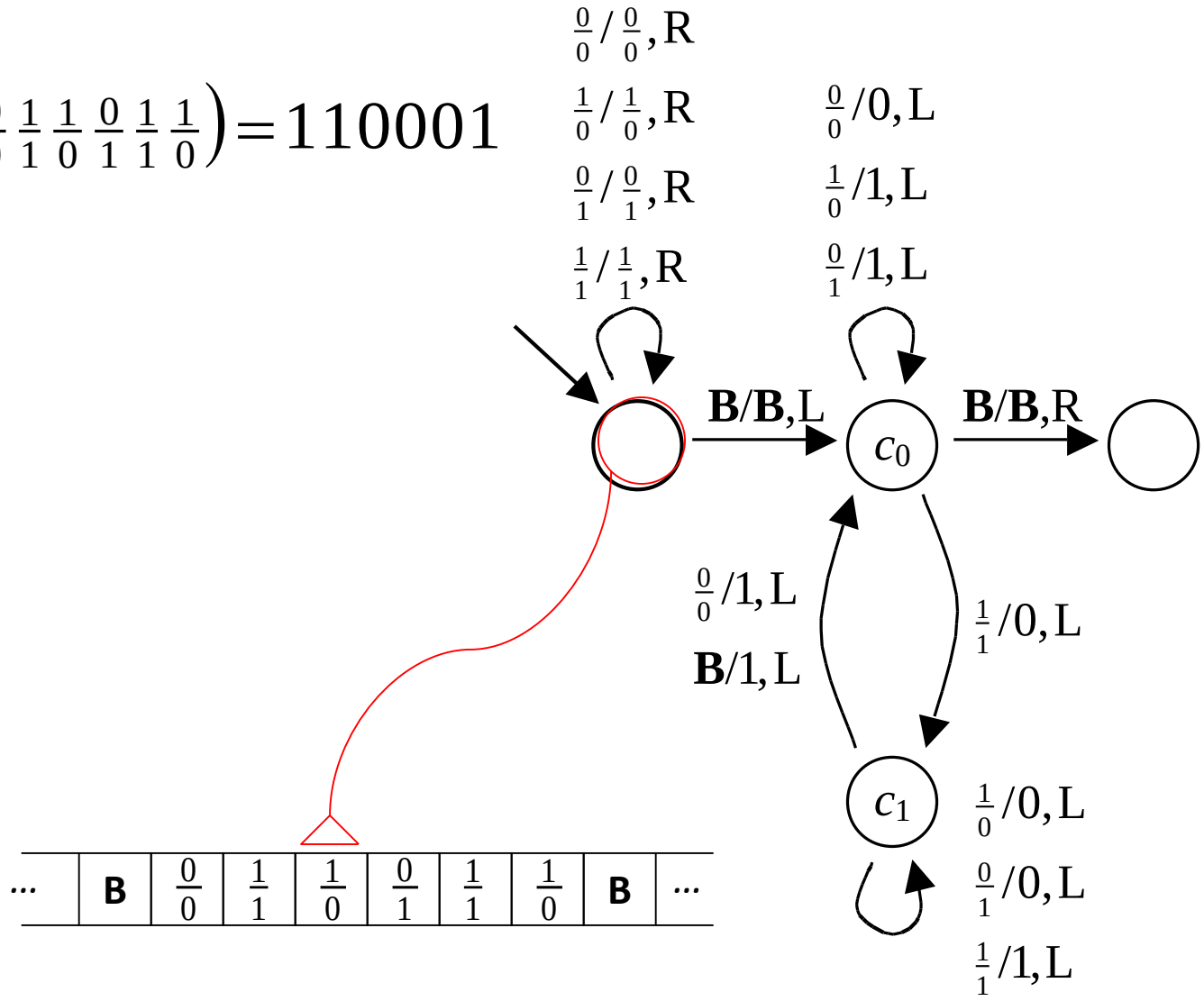
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



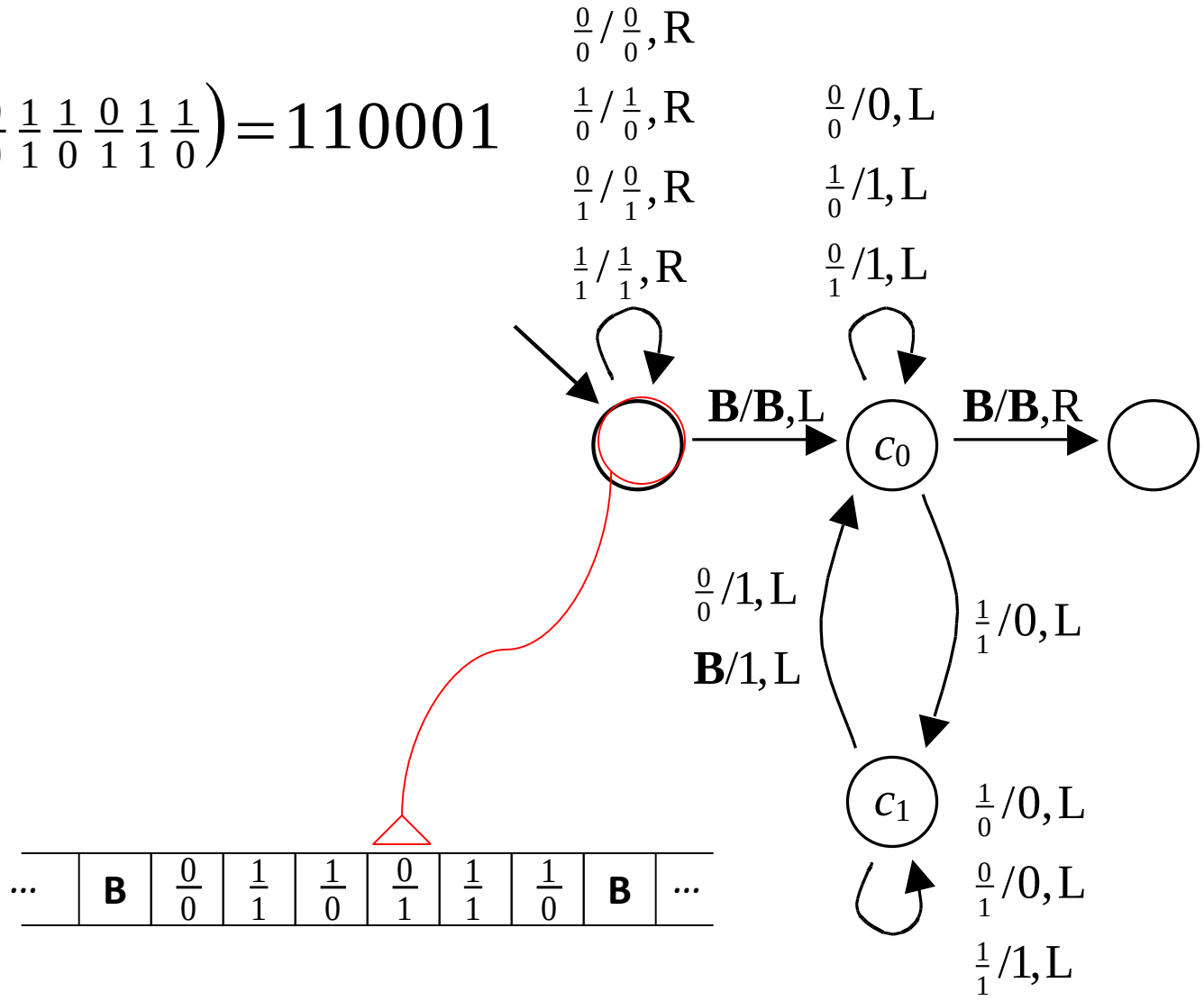
$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$



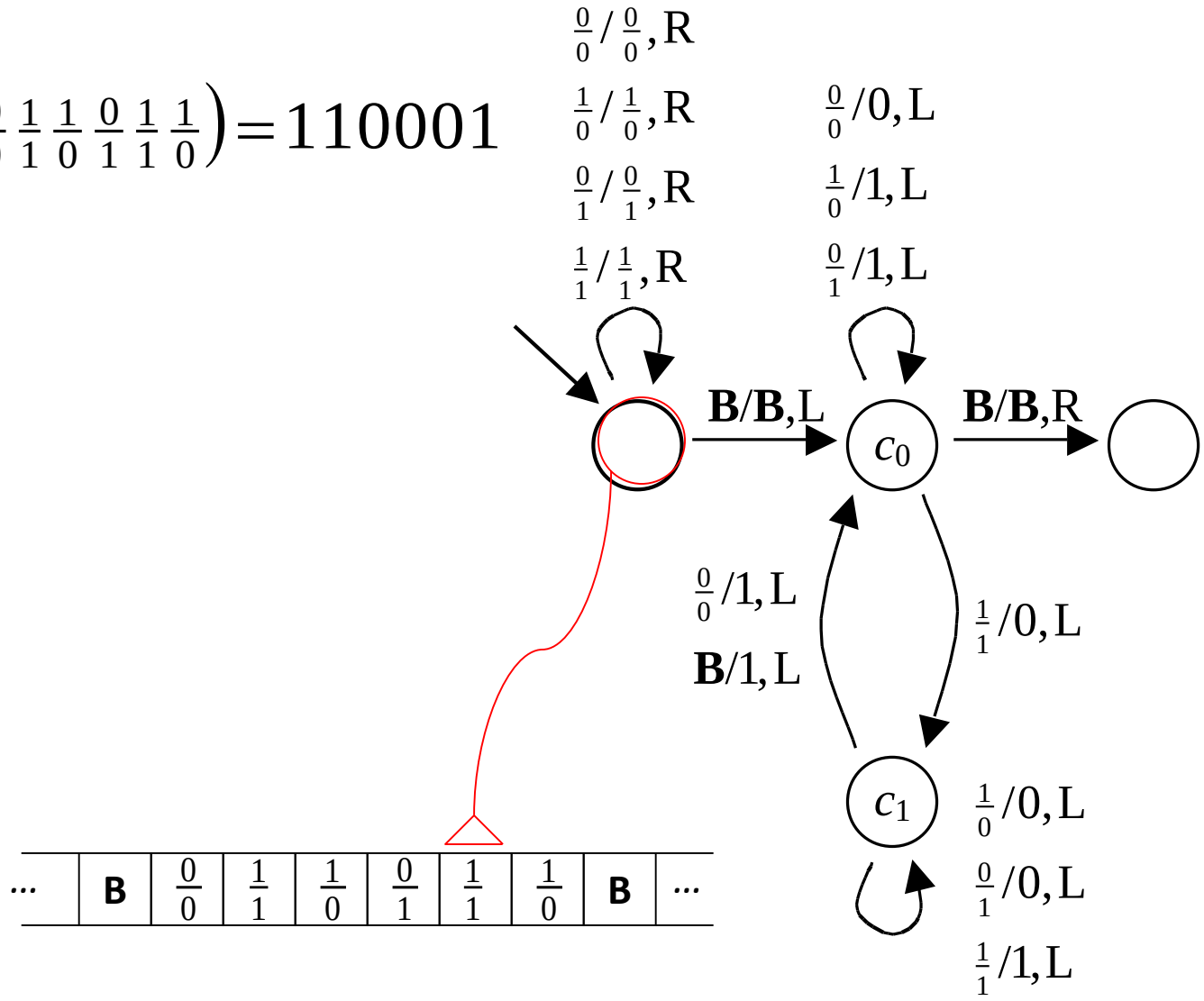
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



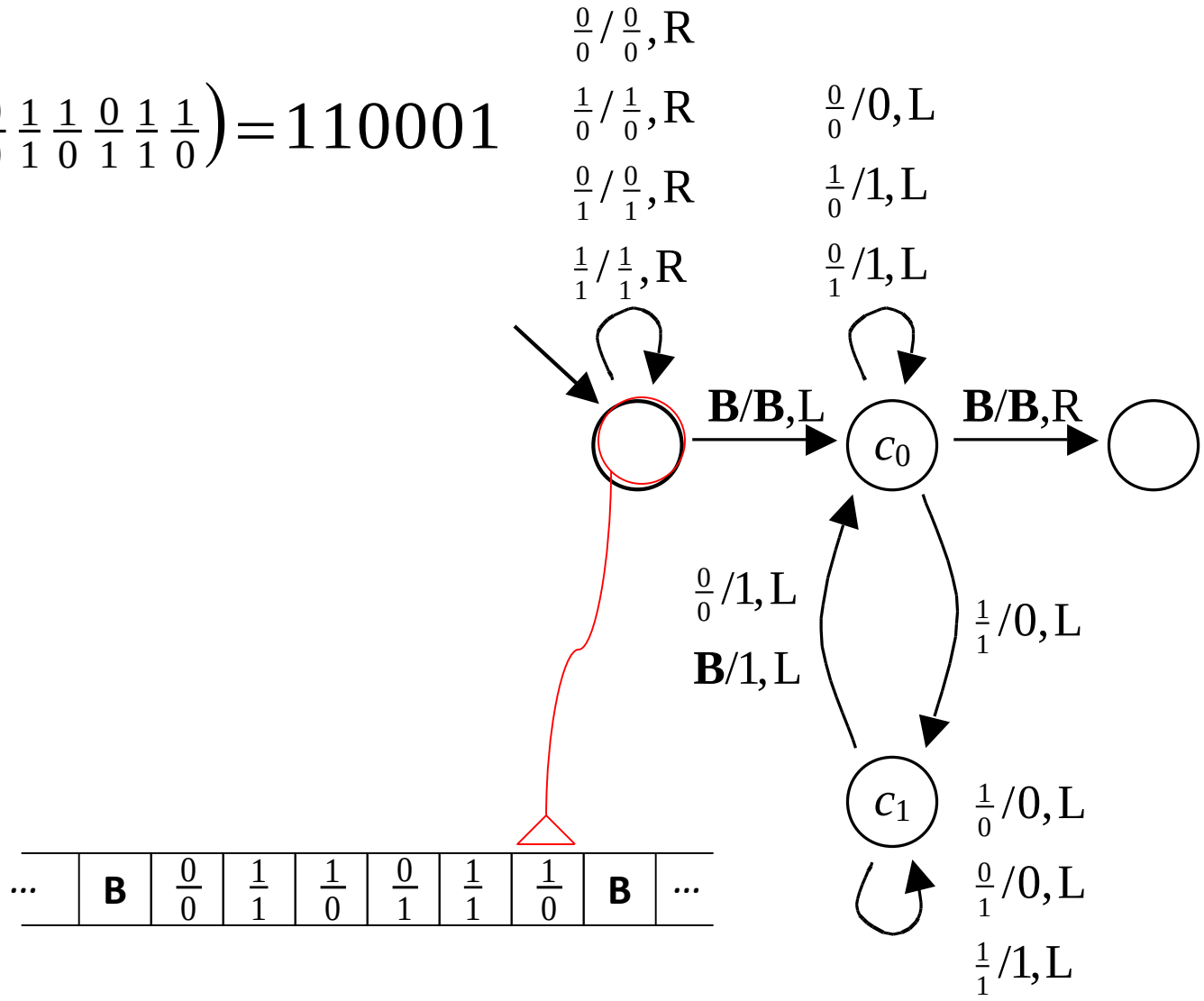
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



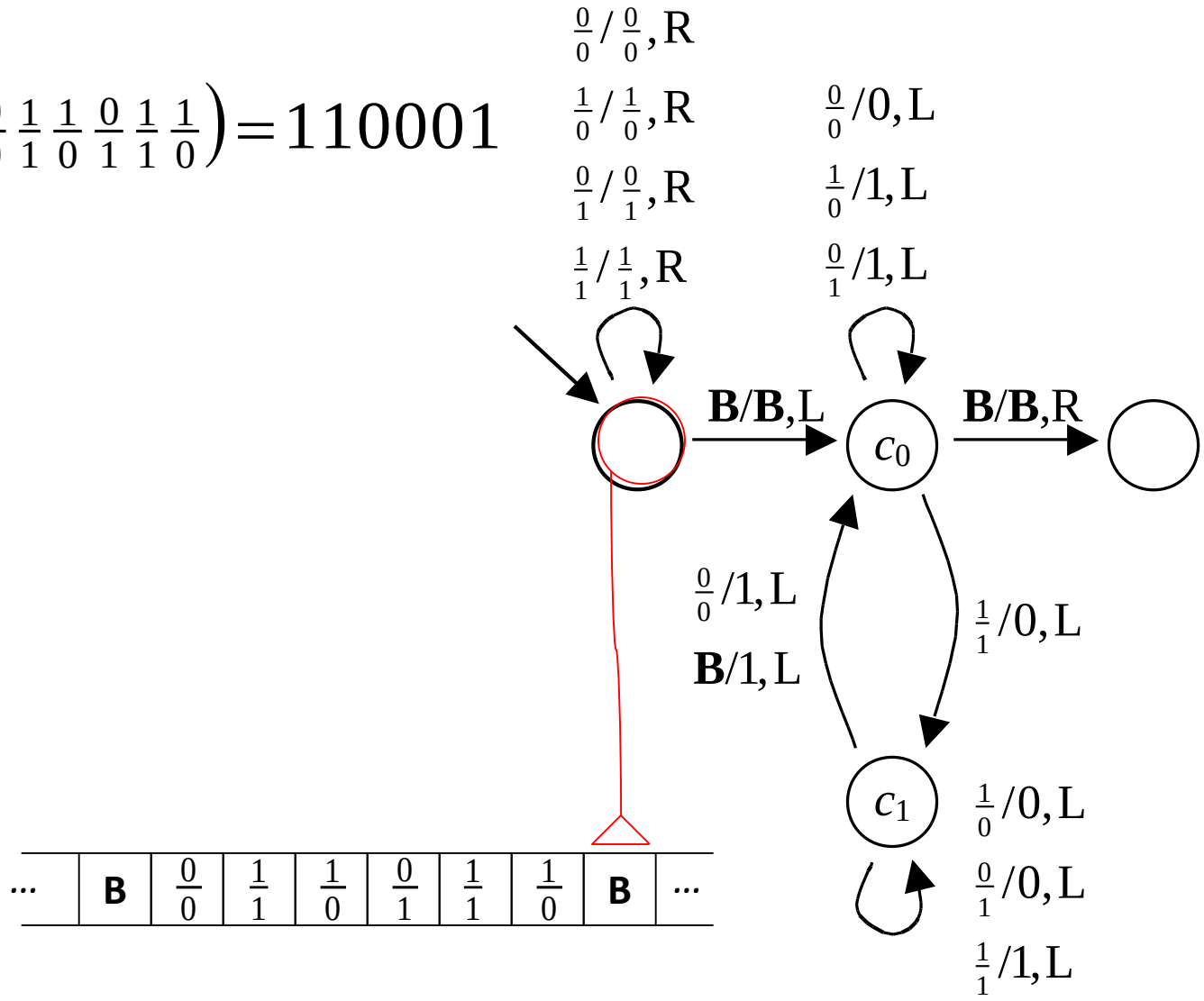
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



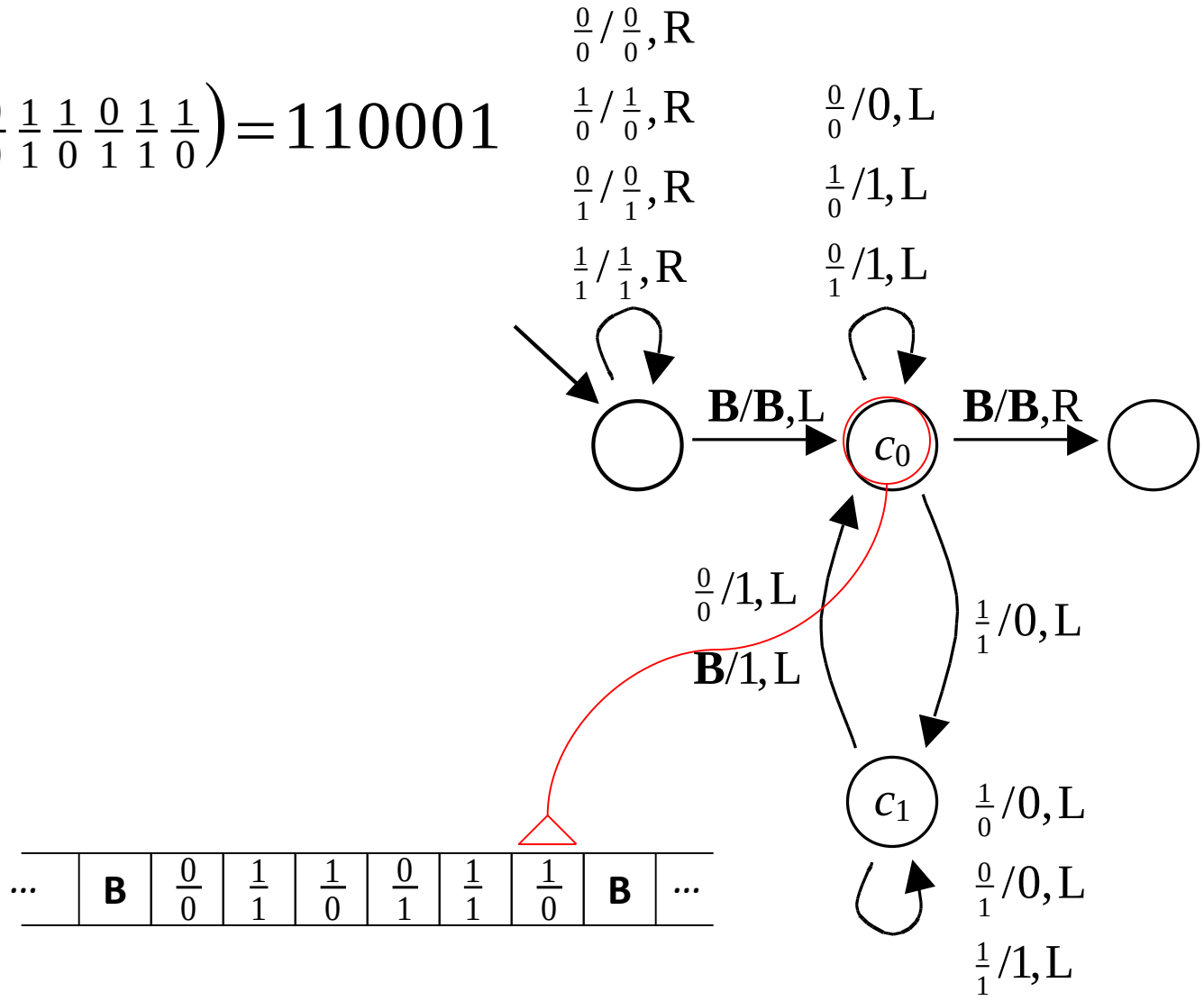
$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$



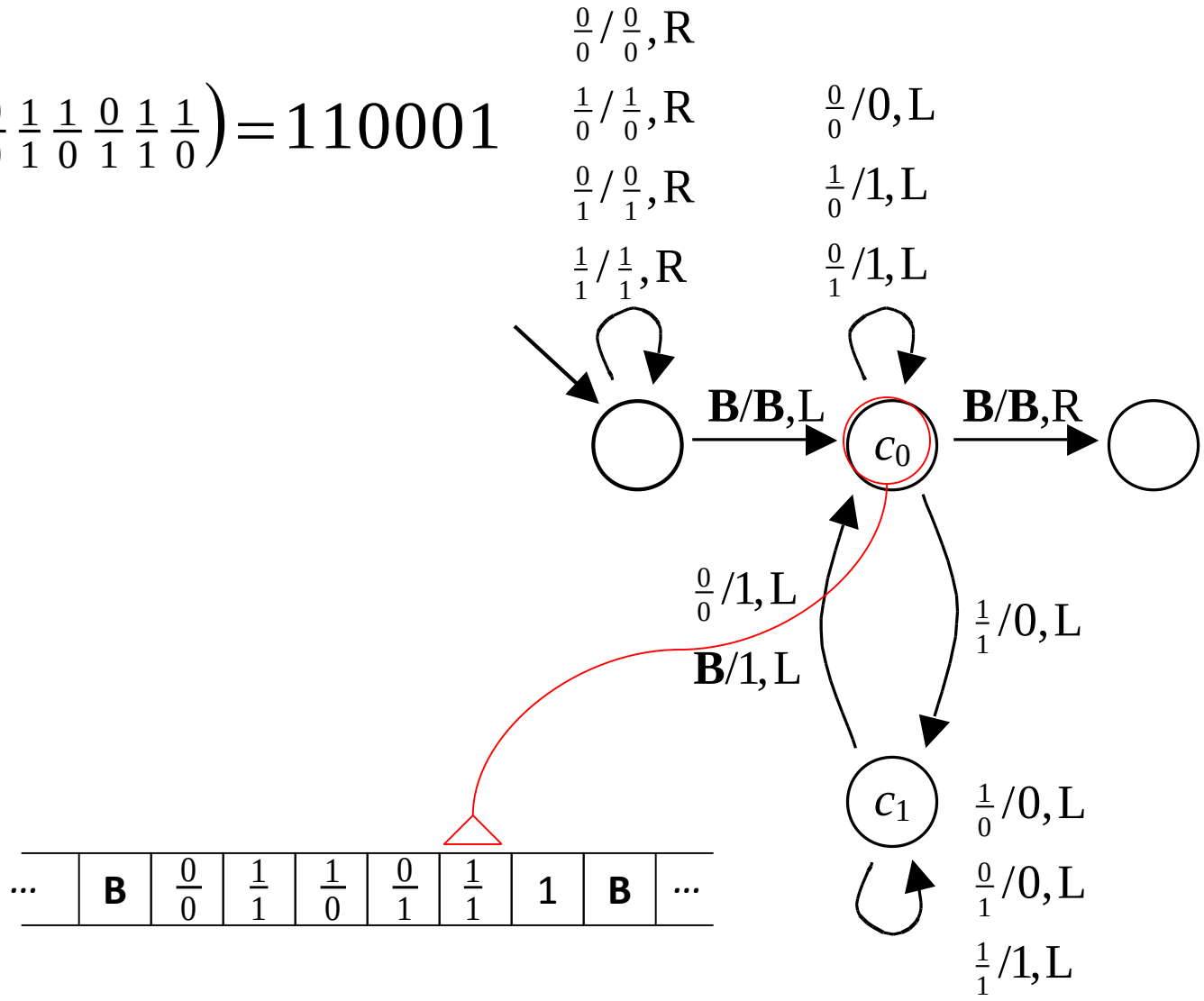
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



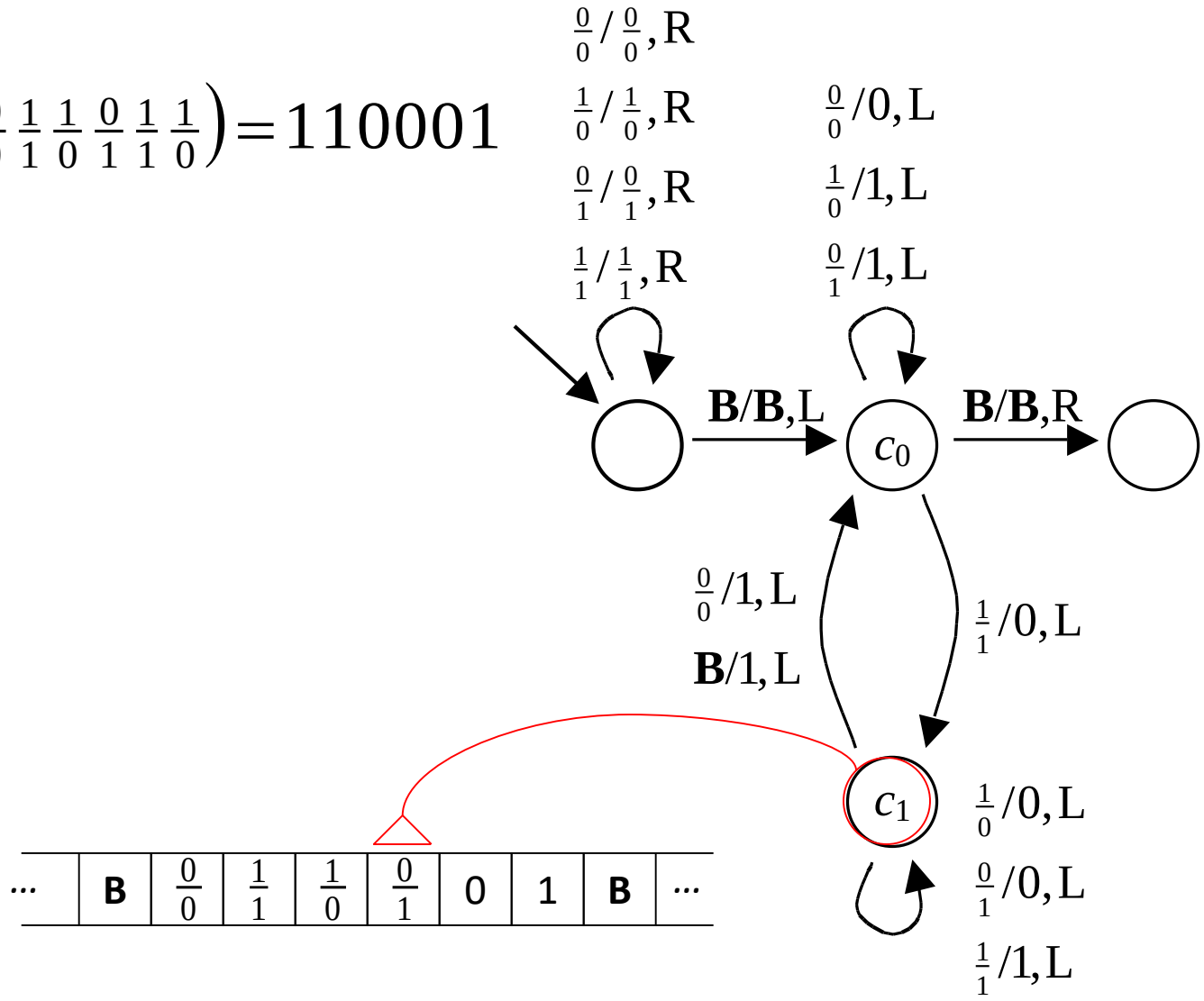
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



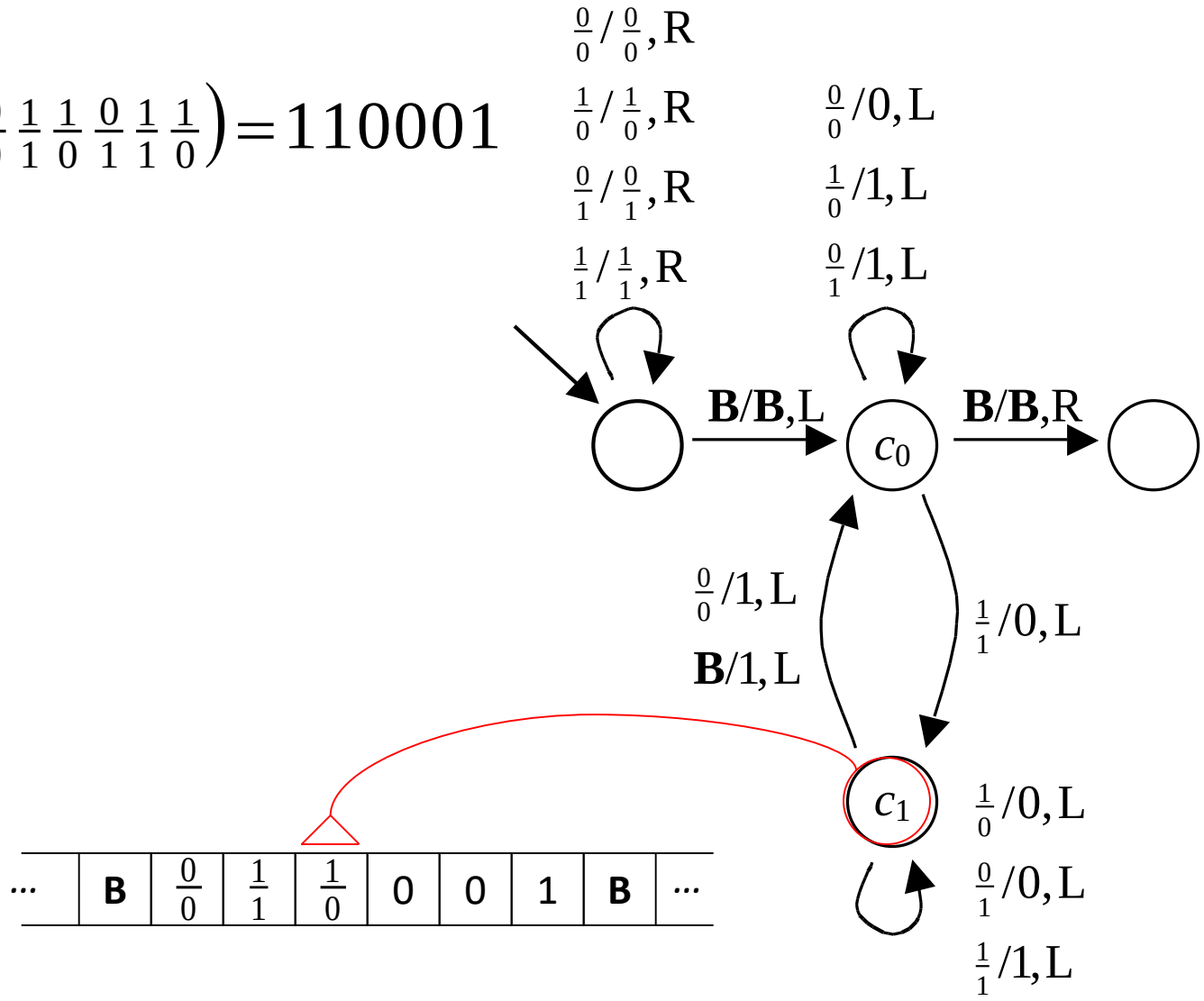
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



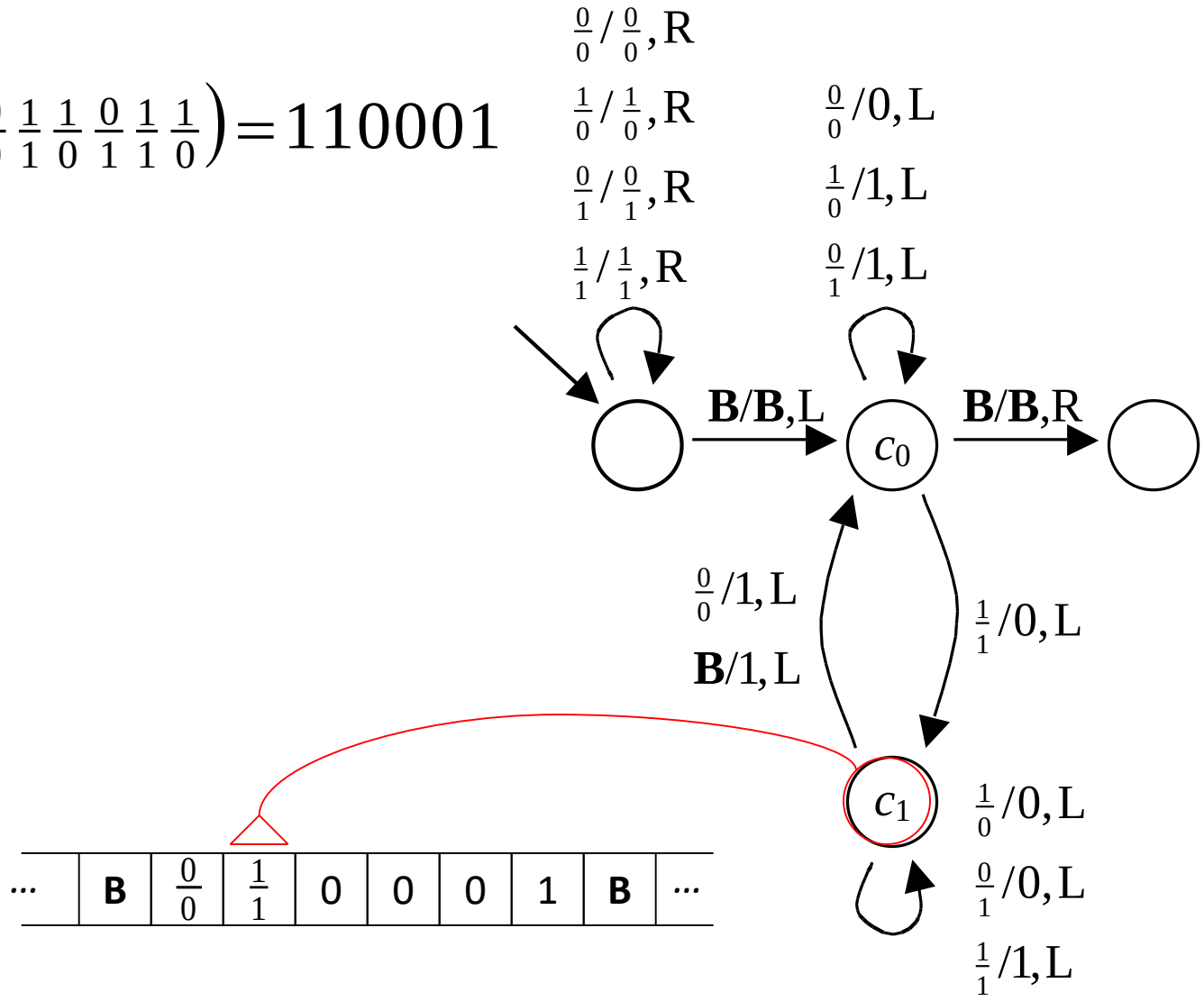
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



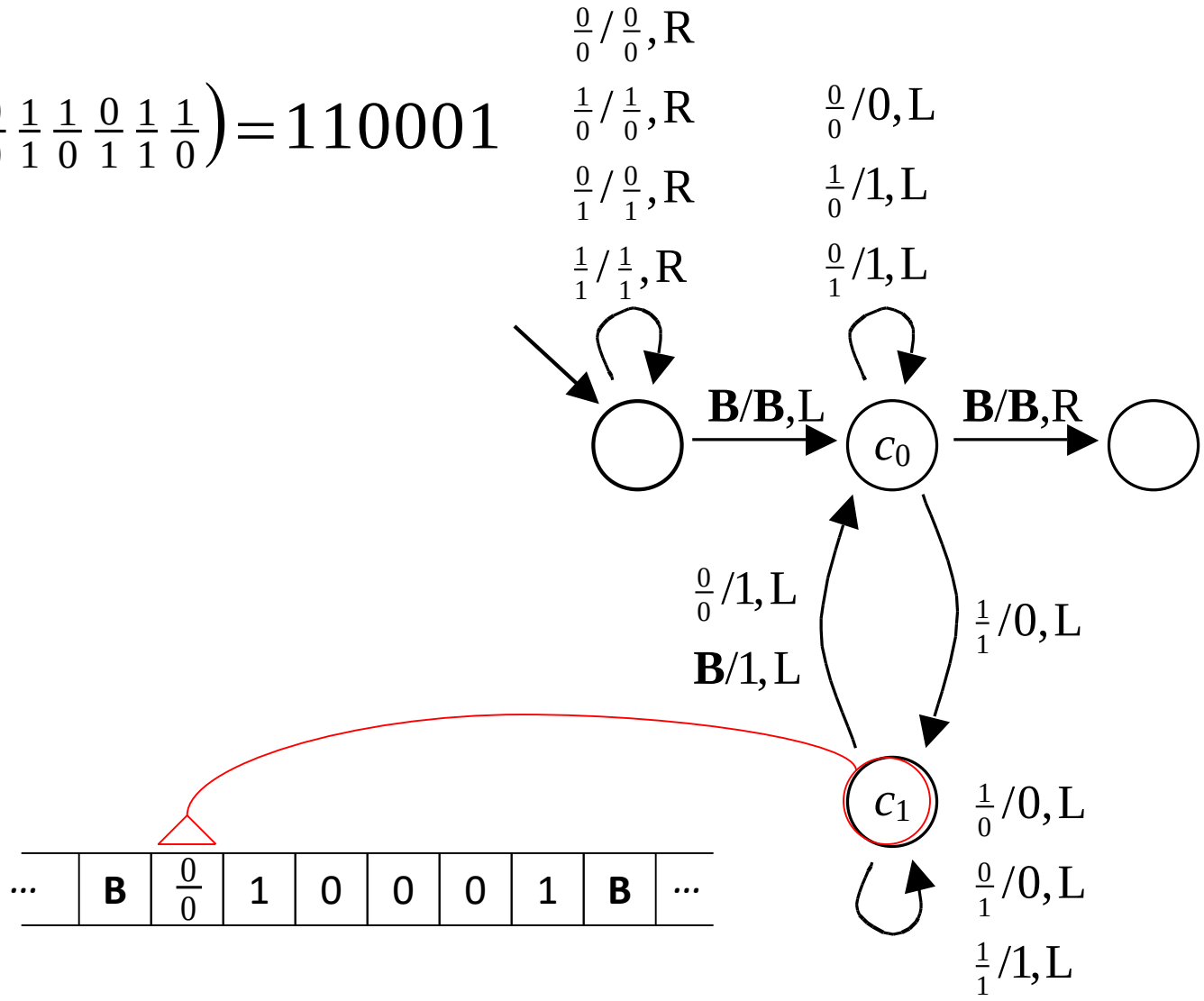
$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$



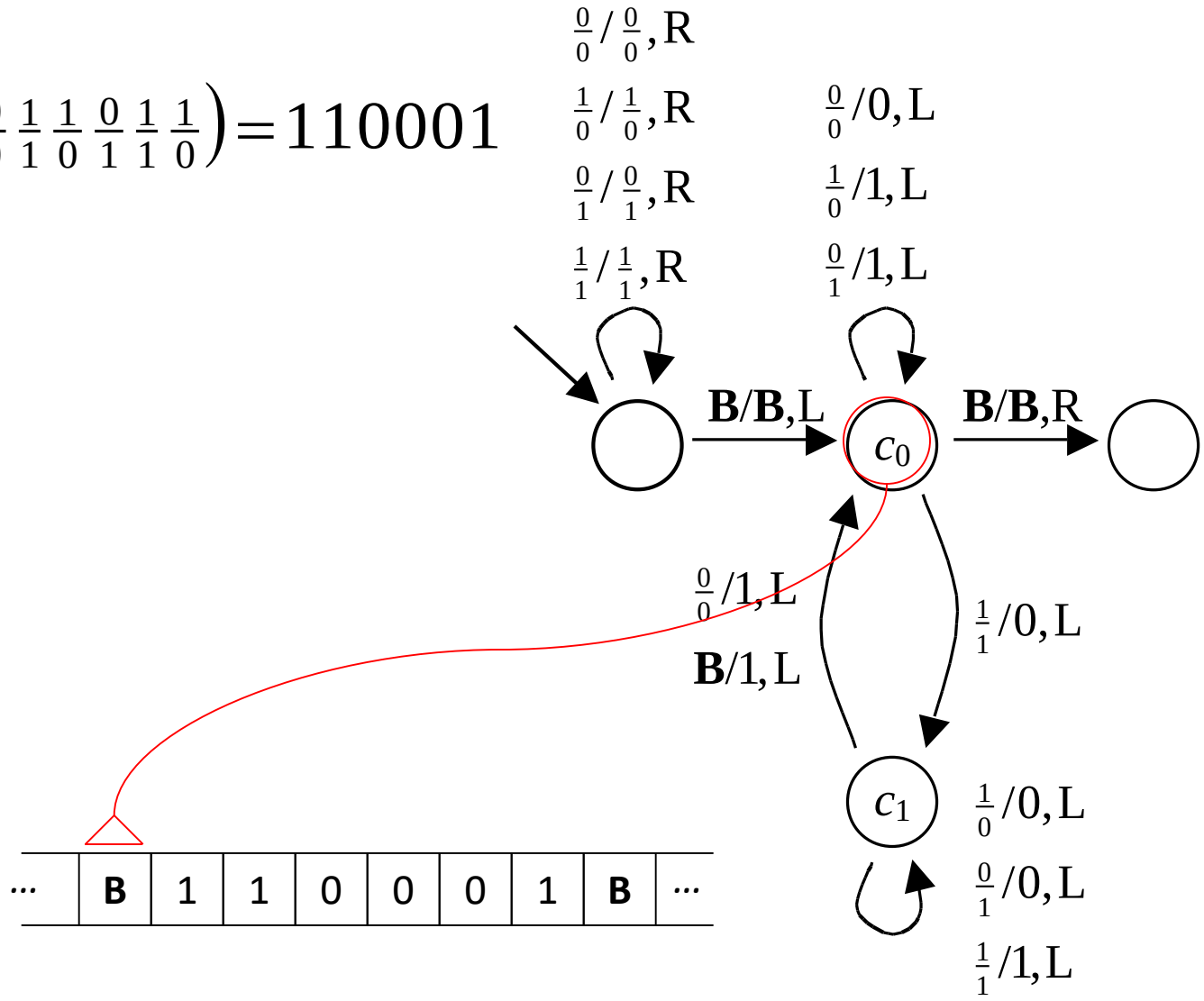
$$add\left(\frac{0}{0}\,\frac{1}{1}\,\frac{1}{0}\,\frac{0}{1}\,\frac{1}{1}\,\frac{1}{0}\right)=110001$$



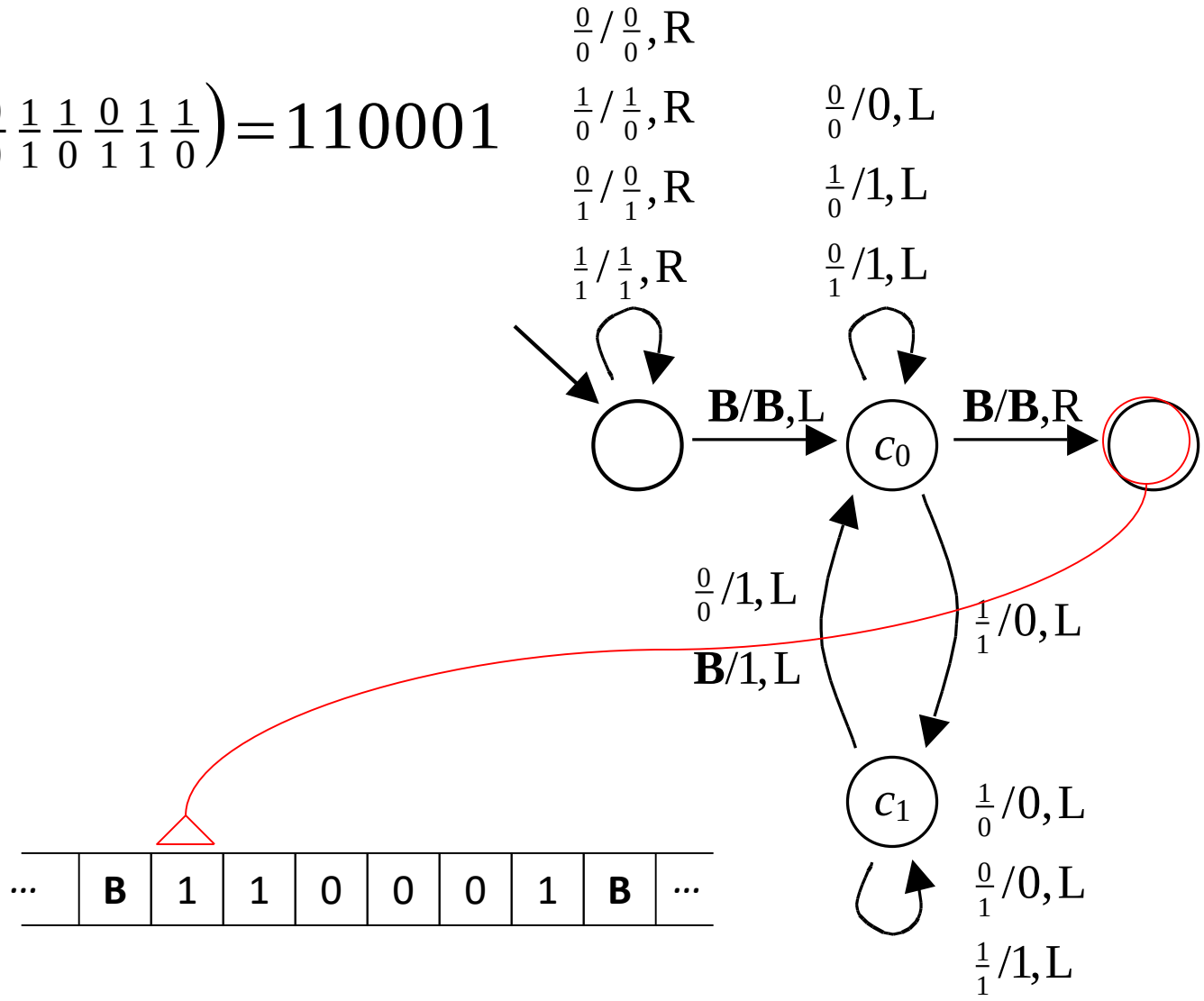
$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$



$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$



$$add\left(\frac{0}{0}\frac{1}{1}\frac{1}{0}\frac{0}{1}\frac{1}{1}\frac{1}{0}\right)=110001$$

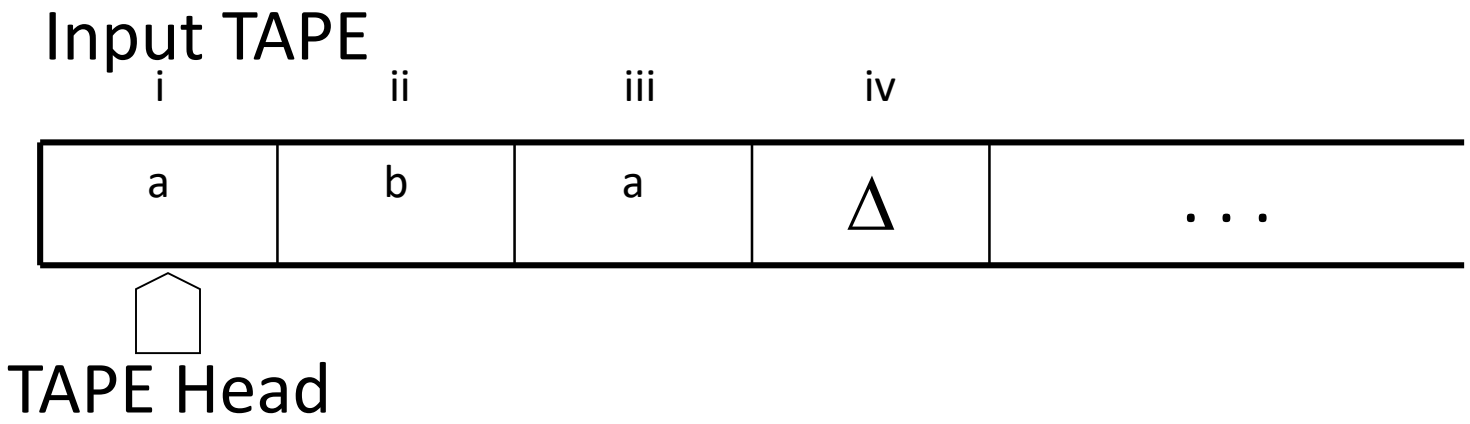


Turing machine

Definition: A Turing machine (TM) consists of the following

1. An alphabet Σ of input letters.
2. An input TAPE partitioned into cells, having infinite many locations in one direction. The input string is placed on the TAPE starting its first letter on the cell i, the rest of the TAPE is initially filled with blanks (Δ's).

Turing machine continued …



3. A tape Head can read the contents of cell on the TAPE in one step. It can replace the character at any cell and can reposition itself to the next cell to the right or to the left of that it has just read.

Turing machine continued …

Initially the TAPE Head is at the cell i. The TAPE Head can't move to the left of cell i. the location of the TAPE Head is denoted by



4. An alphabet Γ of characters that can be printed on the TAPE by the TAPE Head. Γ may include the letters of Σ. Even the TAPE Head can print blank Δ, which means to erase some character from the TAPE.

Turing machine continued …

5. Finite set of states containing exactly one START state and some (may be none) HALT states that cause execution to terminate when the HALT states are entered.
6. A **program** which is the set of rules, which show that which state is to be entered when a letter is read form the TAPE and what character is to be printed. This program is shown by the states connected by directed edges labeled by triplet (letter, letter, direction)

- The first letter (either Δ or from Σ or Γ) is the character the tape head reads from the cell to which it points. Second letter (Δ or from Γ) is what the tape head prints in the cell before it leaves. Third part, direction tells the tape head to move one cell right R or left L.

Note

It may be noted that there may not be any outgoing edge at certain state for certain letter to be read from the TAPE, which creates nondeterminism in Turing machines.

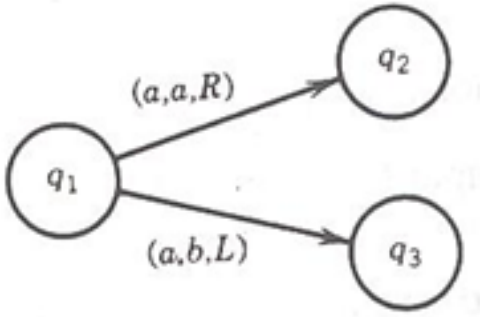
It may also be noted that at certain state, there can’t be more than one out going edges for certain letter to be read from the TAPE. The machine crashes if there is not path for a letter to be read from the TAPE and the corresponding string is supposed to be rejected.

Note continued …

To terminate execution of certain input string successfully, a HALT state must be entered and the corresponding string is supposed to be accepted by the TM.

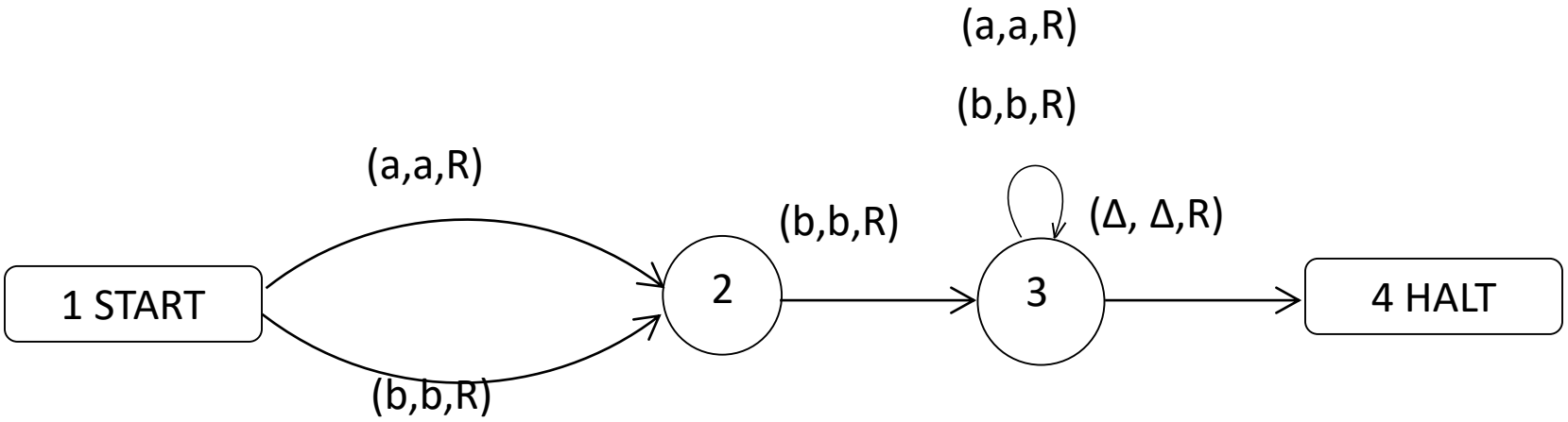
The machine also crashes when the TAPE Head is instructed to move one cell to the left of cell i.

By definition Turing Machines are Deterministic. This means there is no state q, that has two or more edges leaving it labelled with the same first letter as follows:



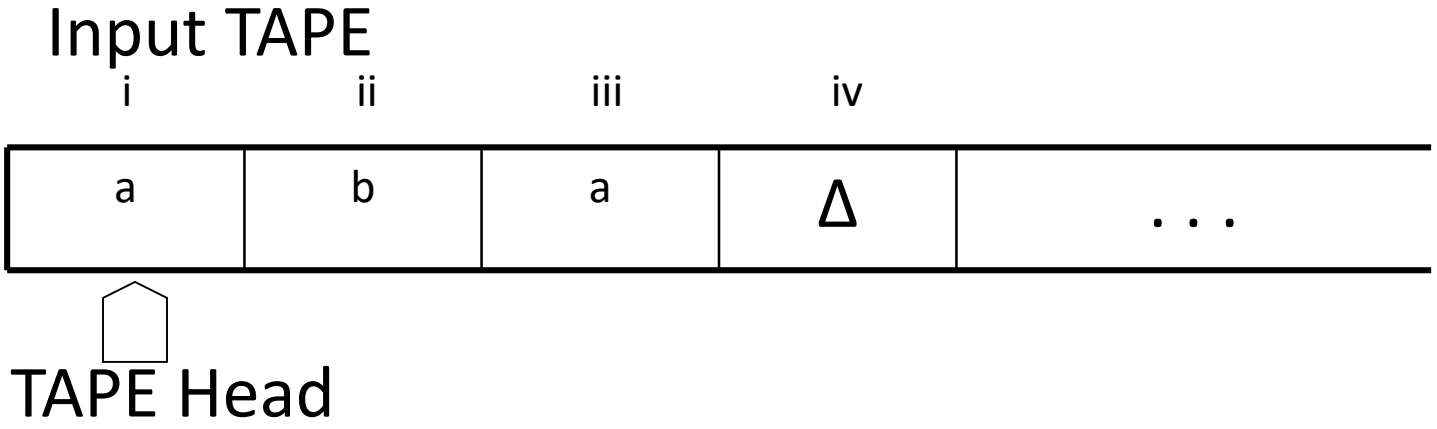
Example

Consider the following Turing machine



Let the input string aba be run over this TM

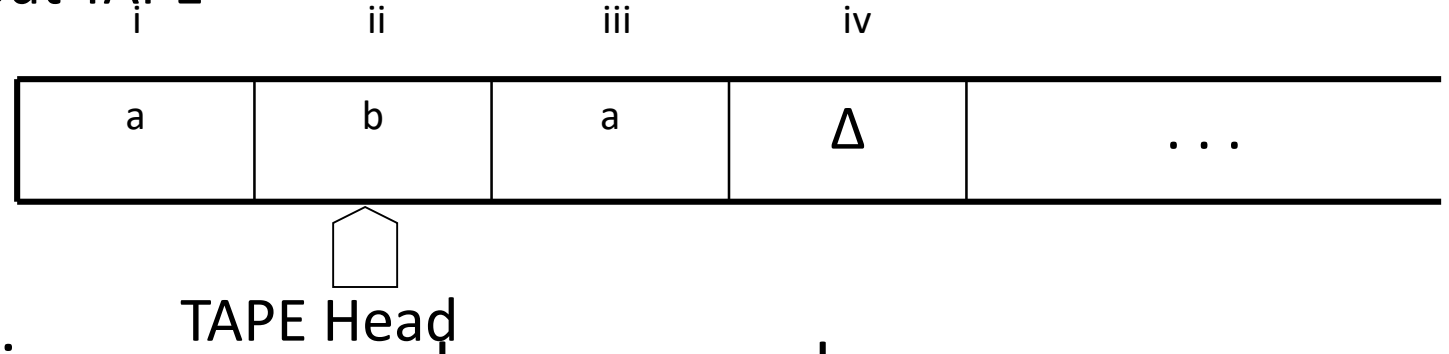
Example continued …



Starting from the START state, reading a form the TAPE and according to the TM program, a will be printed *i.e.* a will be replaced by a and the TAPE Head will be moved one cell to the right.

Which can be seen as

Input TAPE

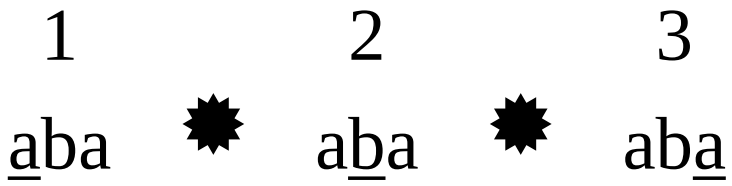


This process can be expressed as

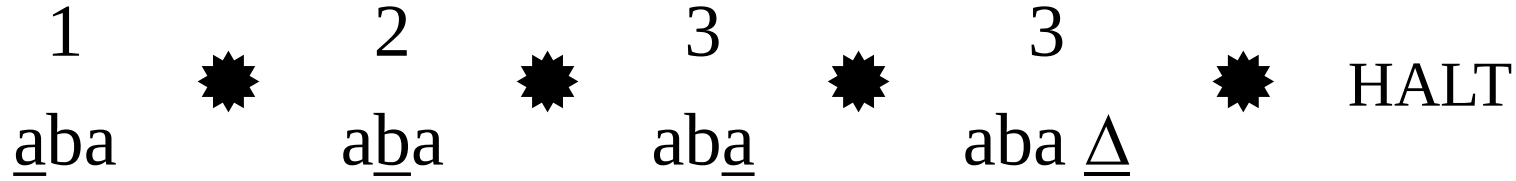
1 2

aba aba

At state 2 reading b, state 3 is entered and the letter b is replaced by b, *i.e.*



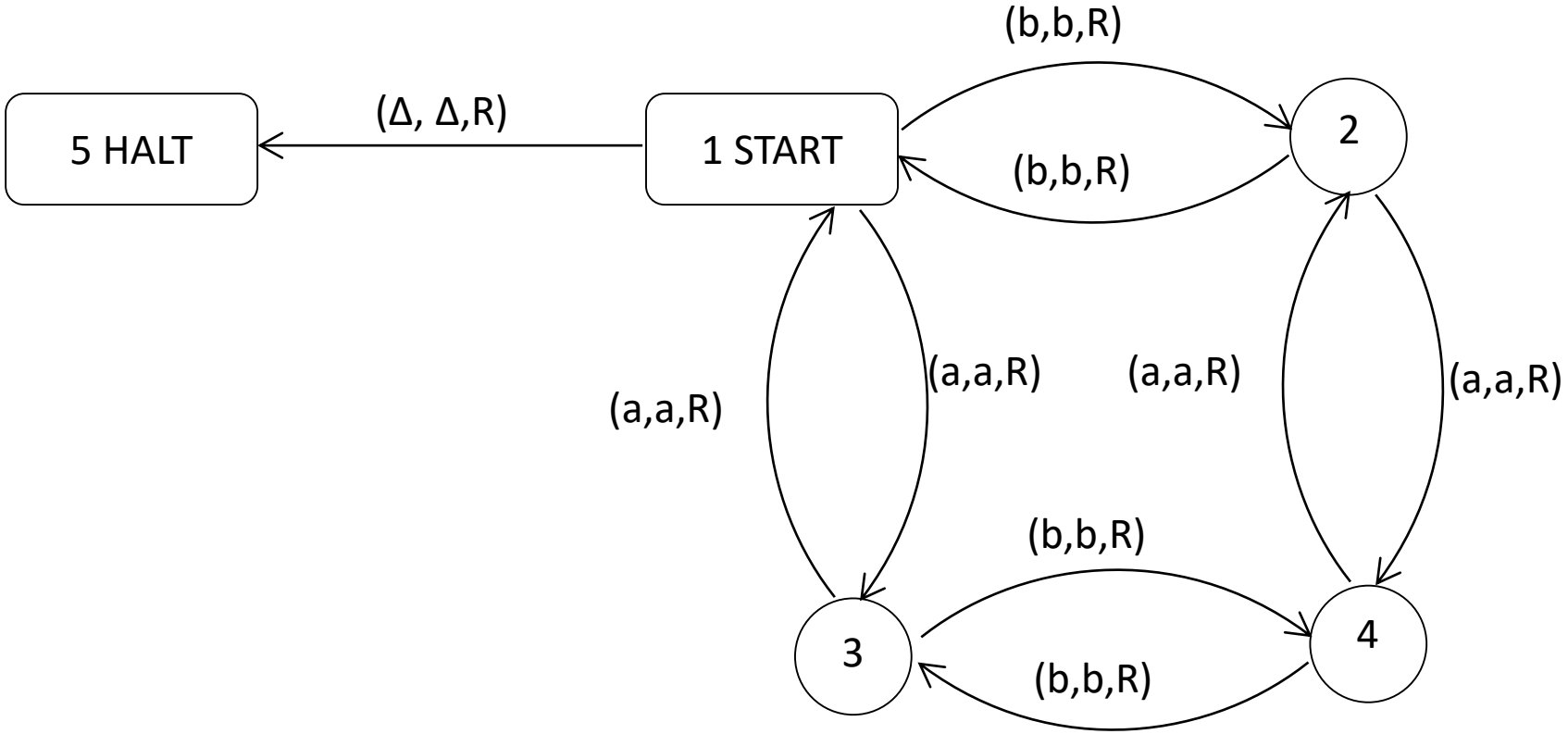
At state 3 reading a, will keep the state of the TM unchanged. Lastly, the blank Δ is read and Δ is replaced by Δ and the HALT state is entered. Which can be expressed as



Which shows that the string aba is accepted by this machine. It can be observed, from the program of the TM, that the machine accepts the language expressed by $(a+b)b(a+b)^*$.

Theorem: Every regular language is accepted by some TM.

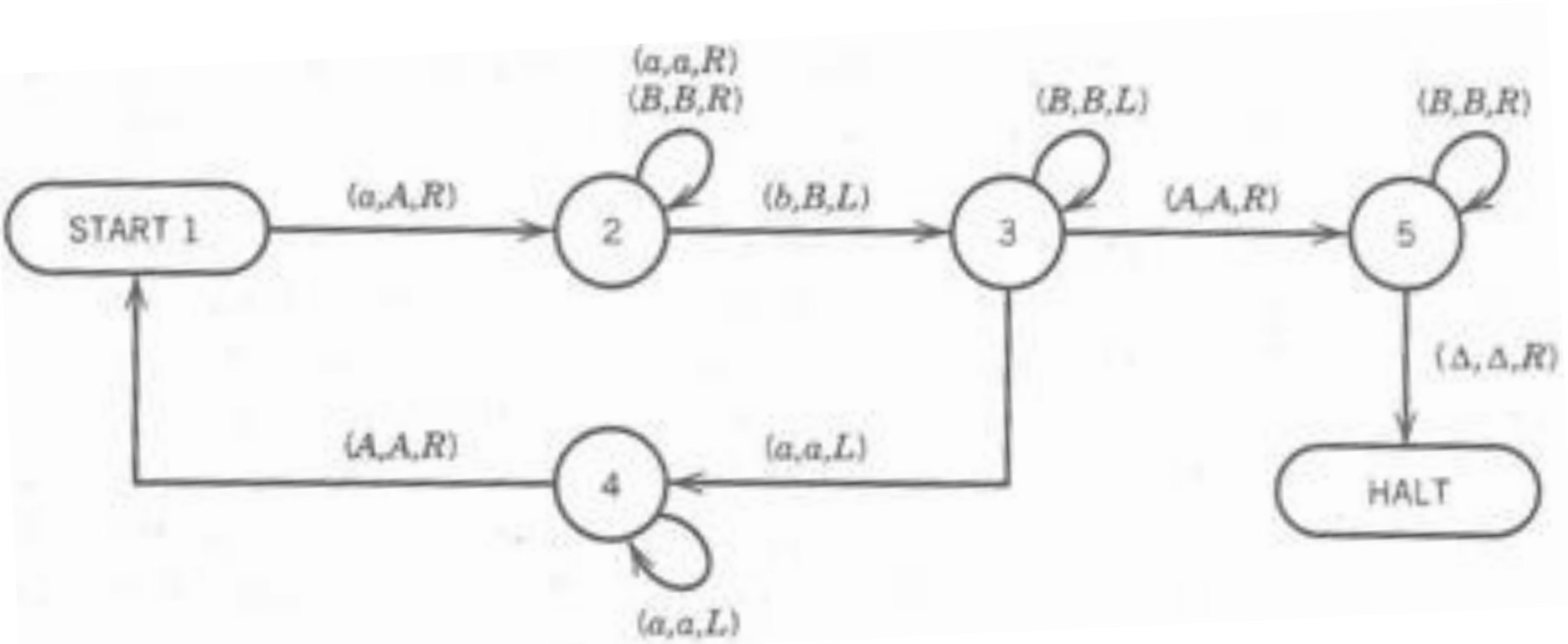
Example: Consider the EVEN-EVEN language. Following is a TM accepting the EVEN-EVEN language.



It may be noted that the above diagram is similar to that of FA corresponding to EVEN-EVEN language. Following is another example

Example

Consider the following TM



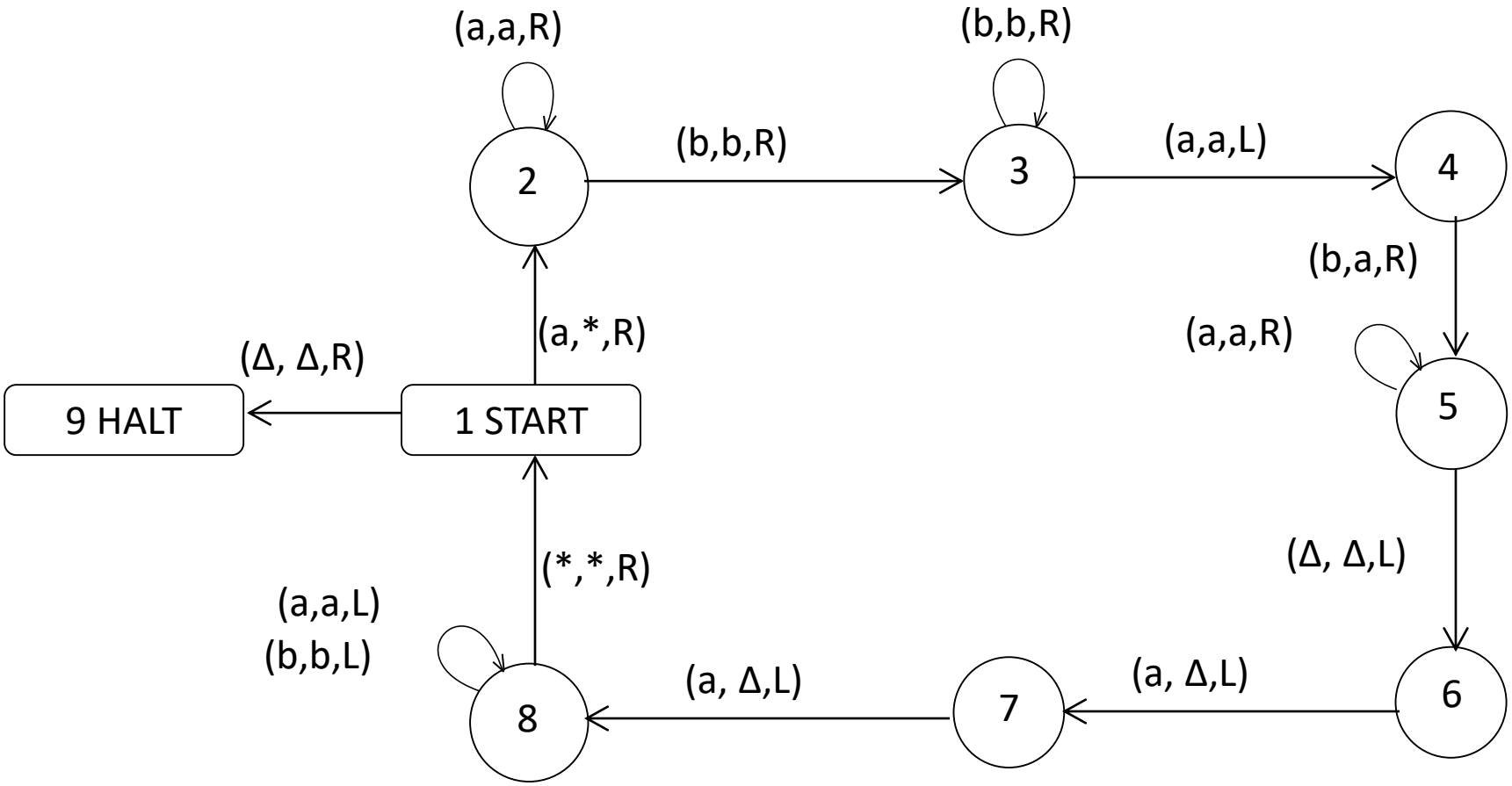
Example

- Execute abb, aabb, aaabbb on that TM
- It accepts language : $\mathbf{a^n b^n}$



Example

Consider the following TM



Example continued …

The string aaabbbaaa can be observed to be accepted by the above TM. It can also be observed that the above TM accepts the non-CFL $\{a^n b^n a^n\}$.

A TM *recognizes* a language iff it accepts all and only those strings in the language

A language L is called Turing-recognizable or recursively enumerable iff some TM recognizes L

A TM *decides* a language L iff it accepts all strings in L and rejects all strings not in L

A language L is called decidable or recursive iff some TM decides L

A language is called Turing-recognizable or recursively enumerable (r.e.) if some TM recognizes it

A language is called decidable or recursive if some TM decides it

